

In this assignment, I implemented the Gaussian Naive Bayes algorithm to classify the spam and non-spam emails. The very first thing I did was get the data and thoroughly shuffled the instances and then split it into two halves i.e., training set and test set. Each of the dataset contains 2,300 instances, out of which 40% are spam, 60% not spam. Further, each feature is independent of all others, I did not standardize the features. I calculated the mean and standard deviation for each of the 57 features in the training set of the values given for each class, followed by replacing the zero standard deviation of any feature with "minimal" standard deviation (e.g. 0.0001) to avoid a divide-by-zero error in Gaussian Naive Bayes. And finally, calculated the confusion matrix, accuracy, precision and recall values.

```
/usr/local/bin/python3.7 /Users/sanjuktadas/Pycharm_Python_Workspace/CS545-P2/p2.py
length of result : 2301

Confusion Matrix:

[[933 452]
 [ 28 888]]

Accuracy : 0.7913950456323338
Precision: 0.6626865671641791
Recall    : 0.9694323144104804

Process finished with exit code 0
```

I got about 79.1% accuracy, 66.2% precision and 96.9% recall values. The confusion matrix shows that for most part it is able to classify spam and non spam emails. However, the accuracy is still not great.

Do you think the attributes here are independent, as assumed by Naive Bayes?

My assumption was that all the attributes are mutually independent in Naive Bayes so that in order to compute the outcome probability we can multiply the class conditional probabilities. However, there can be few dependencies and some of the attributes can be correlated, it is easy to ignore one of the correlated attributes. However, the classifier does not know which attributes are dependent on one another.

Does Naive Bayes do well on this problem in spite of the independence assumption?

From the above results, it can be said that the Naive Bayes does alright. However, a model with 90% accuracy would have been great. This model can be improved by removing redundant data i.e., correlated attributes and use probabilities for feature selection i.e., selection of the data attributes that best characterize a predicted variable. To find the combination of probabilities of different attributes and evaluate their performance for predicting the output variable we could use a search algorithm.